

Infrastructure I need Before Deployment

No HTTPS / TLS termination layer

Fixing issue 7 adds TLS to the Java server directly, but in production you never expose your Java process to the public internet. You need a reverse proxy in front of it — Nginx or Caddy — that handles TLS termination, certificate renewal via Let's Encrypt, and forwards clean HTTP to your Java node internally. If your Java process crashes, Nginx returns a proper 502 instead of a hanging connection.

No process supervision

Right now if your Java node crashes, it stays down until someone manually restarts it. You need `systemd` on Linux to run the node as a service that automatically restarts on failure, starts on boot, and writes logs to a proper location. One config file, but without it a single JVM crash takes down the whole system until you notice.

No log aggregation

Your node currently prints to `stdout`. In production you need structured logs — JSON format, written to files, rotated daily, shipped to a log aggregator like Grafana Loki, Datadog, or even just CloudWatch. When something goes wrong at 2am you need to be able to search logs, not SSH into a server and `tail -f` a file hoping to catch something.

No monitoring or alerting

You have no visibility into whether your node is healthy right now. You need at minimum: uptime monitoring (something like UptimeRobot pinging `/health` every minute and texting you if it's down), JVM heap usage monitoring (Java can run fine then suddenly die when GC can't keep up), disk usage alerts (your `blocks.log` grows forever — you need a warning before the disk fills), and mempool size monitoring (a growing mempool is your early warning for stuck transactions).

No backup strategy

Your entire blockchain state is in `blocks.log`. If that file is corrupted or the disk fails, everything is gone. You need automated daily backups to a separate storage location — S3 or even just a second disk — with a tested restore procedure. "Tested" means you actually restored from backup at least once and confirmed the node came back correctly.

No disaster recovery plan

If your server goes down completely, how long does it take you to get back online? Where is your backup server? Do you have the node configuration documented somewhere outside the server itself? Do you have the API keys, genesis keypair, and peer list stored securely offsite? None of this exists in your current setup.

Security hardening you still need

Firewall rules

Right now every port on your server is likely open to the world. You need a firewall (AWS Security Groups, or `ufw` on Linux) that allows: port 443 for HTTPS, your P2P port only from known peer IPs, and

port 22 for SSH only from your own IP addresses. Everything else blocked. The REST API should never be directly reachable on port 8080 from the public internet.

No input sanitization

Your REST endpoints deserialize JSON directly into Java objects with no size limits. A malicious actor can send a 500MB JSON payload to any endpoint and cause your server to run out of memory trying to deserialize it. Add a maximum request body size limit at the Nginx layer (e.g. `client_max_body_size 1m`) and validate all incoming fields have reasonable length and format before touching them.

No brute force protection

Someone can attempt unlimited API key guesses with no lockout. Add exponential backoff on failed authentication — after 5 failed attempts from an IP, lock it out for increasing periods.

Dependency vulnerabilities

You're using Jackson, Bouncy Castle, and a handful of other libraries. Any of these could have known CVEs. Before deployment, run `mvn dependency:check` with the OWASP dependency check plugin and fix any high or critical vulnerabilities in your dependencies.

No security headers

Your REST API responses have no security headers. Add at minimum: `X-Content-Type-Options: nosniff`, `X-Frame-Options: DENY`, `Strict-Transport-Security` once TLS is in place. These are one-time additions to your `RestServer` response handler.

Compliance and legal requirements

Money Services Business (MSB) license

Your financial model already budgets for this, but it's worth understanding what it means operationally. If SMEChain touches fiat money in any way — even just facilitating payments — you are likely operating as an MSB under FinCEN rules. You need to register with FinCEN, implement a written AML program, file Suspicious Activity Reports when triggered, and keep records for 5 years. This is not optional and the penalties for non-compliance are severe.

KYC is not optional at launch

Your v2 stubs mark KYC as a future feature. But you cannot legally onboard a single paying business without verifying their identity first under Bank Secrecy Act requirements. Before your first real client, you need a real KYC provider integrated — Persona, Jumio, or Stripe Identity are all reasonable starting points. The `SimulatedKycProvider` that approves anyone with a non-blank legal name cannot touch production.

Terms of service and privacy policy

You are storing business invoice data — amounts, counterparties, payment history. This is sensitive commercial data. You need a privacy policy that explains what you store, how long you keep it, who can access it, and how businesses can request deletion. You need terms of service that define liability limits, dispute resolution procedures for platform-level issues, and acceptable use. These need to be reviewed by a lawyer, not drafted by you.

Data residency

If any of your clients are in the EU, GDPR applies. Invoice data with identifiable business information may constitute personal data under GDPR. You need to understand where your data is stored geographically, have a Data Processing Agreement template ready, and have a process for responding to data subject requests within 72 hours.

Operational gaps

No staging environment

Right now you have one environment — your laptop or your production server. You need a separate staging environment that mirrors production exactly, where you can deploy and test changes before they hit real clients. Every bug fix from the list of 30 goes through staging first.

No deployment pipeline

Right now deploying a change means SSHing into a server and running commands manually. You need at minimum a simple CI/CD pipeline — GitHub Actions works fine — that runs your tests, builds the JAR, and deploys to staging automatically on every push to main. Production deployments should be a deliberate manual trigger after staging validation.

No database for the application layer

Your financial model mentions a dashboard that SMEs use to check invoice status. That dashboard cannot query `blocks.log` directly — parsing the entire chain log for every page load is not viable. You need a SQL database (Postgres is the right choice) that mirrors invoice state, updated every time a block is committed. This is the async mirroring problem from your original list — you need to build it carefully with the consistency issues in mind.

No client onboarding flow

How does a new business actually join SMEChain right now? The answer is: they can't, without you manually setting things up for them. You need a self-service onboarding flow: business registers, goes through KYC, gets a wallet created, gets an API key, gets documentation. None of this exists.

No SDK or client library

Real businesses are not going to call `curl POST /tx/issueInvoice` manually. You need a client library — at minimum a JavaScript/TypeScript SDK and ideally a simple web dashboard — that abstracts away the transaction signing, key management, and API calls. This is probably as much work as fixing the 30 bugs.

No API versioning

Your REST endpoints have no version prefix (`/v1/tx/issueInvoice`). The moment you deploy to a real client and then need to change an endpoint's behavior, you will break them. Add `/v1/` prefixes to every endpoint now, before anyone depends on the current paths.

No documentation

There is no API documentation, no integration guide, no runbook for operators, and no architecture document that a new engineer could read to understand the system. Before you onboard your first client you need at minimum: API reference docs (Swagger/OpenAPI generated from your endpoints), a

quickstart guide for new businesses, and an operator runbook covering startup, shutdown, backup, and common failure scenarios.

The honest priority sequence

Before you fix a single one of the 30 bugs, do these first because they are pre-conditions for everything else:

1. Get a lawyer to review your MSB obligations and draft your terms of service
2. Set up your server properly — Nginx, systemd, firewall, SSH hardening
3. Set up monitoring and backups
4. Set up a staging environment and basic CI/CD pipeline
5. Integrate a real KYC provider

Then work through the 30 fixes in the priority order already given.

Then build the client-facing layer — dashboard, SDK, onboarding flow.

Then do a security penetration test by a third party before you go live with real money.

API / Network layer (RestServer.java ,
P2PServer.java)

1. No authentication/authorization on any REST endpoint — `/mine` , `/tx/submit` , `/peers/add` , `/wallet/new` , `/export/snapshot` are open to anyone reachable.
2. Plaintext HTTP — `HttpServer.create(...)` has no TLS; all traffic (including private keys) travels unencrypted.
3. Server generates private keys and returns them in plaintext via `/wallet/new` .
4. Clients must POST raw private keys to `/tx/issueInvoice` and `/tx/simple` so the server can sign — the server sees/could log/leak private keys. Signing must happen client-side in production.
5. Internal exception details leaked to callers — `writeJson(ex, 500, Map.of("error","server_error","detail", e.getMessage()))` exposes stack trace/internal messages.
6. No input validation — `amountCents` , `dueDateEpochSec` , `feeMicrounits` are cast from JSON with no bounds/sign/null checks; a negative amount or garbage due-date will flow straight into a signed transaction.
7. Unbounded request bodies — `readJsonMap` / `readJson` don't cap request size, so large payloads can exhaust memory (DoS).
8. No rate limiting on API or P2P — a single peer/client can spam `/mine` , `/tx/submit` , or P2P messages.
9. Dead/half-implemented async "DB mirror" code in `RestServer` — `LinkedBlockingQueue` fed via `offer()` (silently drops on full, no backpressure) with a consumer thread that does nothing (`// place database updates here`) — looks like leftover/placeholder code that shouldn't ship as-is.
10. P2P layer has zero authentication between nodes — any TCP client can connect and inject `NEW_TX` / `NEW_BLOCK` messages (`P2PServer.handleConn`), and messages aren't signed/verified at the transport level, no peer identity/allowlist.
11. P2P messages deserialized directly via Jackson `convertValue` from untrusted network input with only a try/catch-and-ignore — no schema/size validation, potential for malformed-data DoS.
12. No connection limits/timeouts on P2P `ServerSocket.accept()` loop — unbounded thread-per-connection via `Executors.newCachedThreadPool()` is vulnerable to connection-flood resource exhaustion.

Crypto / key handling (`KeyUtil.java` , `Transaction.java` , `AuthoritySigner.java`)

13. No secure key storage — private keys only ever exist as base64 strings passed around in memory/JSON; no keystore, HSM, or encrypted-at-rest storage.
14. `businessIdFromPubKey` truncates a SHA-256 hex hash to 40 chars ("keep it simple") — reduces collision resistance vs. a proper identifier scheme.

15. Exceptions swallowed silently in `verifySignature()` , `verifyBlockSignature()` — any parsing error is treated the same as an invalid signature, making it hard to distinguish malformed input from actual forgery attempts (harder to detect/alert on attacks).

16. Order-of-operations bug in `/tx/issueInvoice` : `KeyUtil.pubKeyFromB64("")` is called before the blank-check, so an empty key throws an unhandled decode exception instead of the intended validation error.

Storage (`BlockStore.java`)

17. Plaintext append-only log file, no integrity protection (no checksums/signing of the log itself), no encryption at rest, no access controls on the data directory.

18. File I/O has no corruption/crash-recovery handling (e.g., partial writes on crash mid- append).

Fixes for deployment:

Critical Bugs

Issue 1 — ~~Write lock held during mining~~

~~Split `mineBlock()` into two phases. Phase one: grab the read lock, copy the current tip and state, release the lock. Phase two: run PoW completely outside any lock — this is the slow CPU loop that can take seconds. Phase three: grab the write lock only at the very end to commit the finished block. The node stays fully responsive during the entire mining process. Users can submit transactions, peer blocks come in, everything works normally.~~

Issue 2 — Storage race condition

Delete the `Files.size(logFile)` call entirely. Instead open a `FileChannel` in append mode, write the block, then call `channel.position()` after writing to get the exact byte offset the block was written at. That offset is what you store in the index. You're getting the position from the stream itself after the fact, not guessing it before the write.

Issue 3 — Silent state corruption

Replace the empty catch block in `rebuildState()` with actual logic. At minimum: log the block height, the `txId`, and the exception message. In production: throw a `ChainCorruptionException` that halts startup entirely and writes an alert. A corrupted chain that you know about is infinitely better than a corrupted chain that silently serves wrong data.

Issue 4 — Double state application in `tryAddBlock`

Refactor `tryAddBlock()` to do the state application once, then pass the resulting validated state directly into `BlockValidator` as a parameter instead of having `BlockValidator` start from scratch. `BlockValidator` uses the pre-built state rather than re-deriving it.

Issue 5 — Redundant double validation in `mineBlock`

Remove the `BlockValidator.validateBlock()` call from `mineBlock()`. You already validated every transaction before mining. Trust that. Reserve `BlockValidator` exclusively for blocks received from external peers who you don't trust. Your own mined blocks don't need to be validated against themselves.

Security Vulnerabilities

Issue 6 — No replay protection

Add a `nonce` field to every transaction — a simple incrementing integer per sender. The `State` object tracks the last seen nonce for every `businessId`. When a transaction arrives, reject it if its nonce isn't exactly `lastNonce + 1`. This means even if someone captures a signed transaction and replays it later, the nonce will be wrong and the node rejects it. This is exactly how Ethereum handles replay protection.

Issue 7 — Private keys over plain HTTP

Two things. First, add TLS to `RestServer` using Java's built-in `SSLContext` with a Let's Encrypt certificate — free, automated, takes an afternoon to set up. Second, change the `/wallet/new` endpoint: instead of generating keys server-side and sending the private key over the wire, generate the keypair client-side in the browser using the Web Crypto API and only send the public key to the server for registration. The private key never leaves the user's device.

Issue 8 — No authentication

Add an API key middleware layer to `RestServer`. Every request must include an `X-API-Key` header. On startup the node generates a master API key and writes it to a local file only the operator can read. For user-facing endpoints, issue per-business API keys stored alongside their business registration. Add rate limiting per key using a simple token bucket — a `ConcurrentHashMap<String, AtomicInteger>` of request counts per minute, reset on a scheduled timer.

Issue 9 — `/peers/add` DDoS vector

Two changes. First, put `/peers/add` behind admin-only authentication — a separate admin API key. Second, when a peer is added, immediately open a TCP connection and send a `HELLO` message. If you

don't get a valid SMEChain HELLO response back, reject the peer and don't add it to the broadcast list. You're verifying the peer is actually running your software before trusting it.

Issue 10 — Anyone can resolve disputes

Add a `resolverBusinessId` field to `InvoiceRecord`. In the `RESOLVE_DISPUTE` handler in `StateMachine`, add three checks: the outcome string must be one of `"issuer_wins"`, `"counterparty_wins"`, or `"mutual_agreement"`. The sender's `businessId` must not equal `issuerBusinessId`. The sender's `businessId` must not equal `counterpartyBusinessId`. If any check fails, throw `StateTransitionException`. For v2, you add a designated arbitrator registry — only pre-approved arbitrator addresses can resolve.

Issue 11 — businessId truncated SHA-256

Either implement the stated algorithm properly — `RIPEMD160 (SHA256 (pubKey))` using Bouncy Castle which is already a standard Java crypto library — or update every comment and document that says "Bitcoin-style" to accurately describe what you're doing. Pick one. The inconsistency between comment and code is what kills you in an audit.

Issue 12 — Genesis key discarded

Create a `genesis-keypair.json` file written to the `dataDir` on first startup containing the Base64 public and private keys used to sign the genesis transaction. Protect that file with filesystem permissions (`chmod 600`). Document it as a required backup artifact. Now you can always prove genesis authorship.

Concurrency Problems

Issue 13 — getHeight() bypasses lock

One line fix. Inside `Node.getHeight()`, wrap the `store.height()` call with `lock.readLock().lock()` and `lock.readLock().unlock()` in a finally block. Exactly the same pattern used by every other method on the `Node` class. This was simply missed.

Issue 14 — Mempool snapshot race

After PoW completes and before committing, filter the transaction list one more time. For each transaction in your snapshot, check if it's still in the mempool using `mempool.contains(txId)`. If it was removed while you were mining (because a peer block took it), skip it. Commit only the transactions that are still pending. Your block might be smaller than planned but it will always be valid.

Issue 15 — Unbounded thread pool

Replace `Executors.newCachedThreadPool()` with `new ThreadPoolExecutor(10, 100, 60L, TimeUnit.SECONDS, new LinkedBlockingQueue<>(500), new ThreadPoolExecutor.CallerRunsPolicy())`. This gives you a minimum of 10 threads, a maximum of 100, a queue of 500 pending connections, and a rejection policy that slows down the caller rather than crashing. You can tune the numbers — the key is there is now a hard ceiling.

Protocol & Design Problems

Issue 16 — No fork handling

This is the biggest engineering task on the list. You need three things. First, store all known chain tips not just the longest one — a `Map<String, Block>` of `blockHash` to `block`. Second, implement a `reorganize()` method: when a peer's chain is longer than yours, walk back to the common ancestor, reverse all state transitions from your fork, then re-apply all blocks from the peer's fork. Third, use total accumulated difficulty (sum of all block difficulties) not just height as your fork choice rule — this is more attack-resistant. Plan a full sprint for this one.

Issue 17 — Empty blocks

Two-line fix in `mineBlock()`. Before running PoW, check if `(selectedTxns.isEmpty())` throw new `IllegalStateException("No transactions to mine")`. The REST endpoint returns a 400 with a clear message. Miners only mine when there's actual work to do.

Issue 18 — Mempool never evicts

Add a `submittedAt` timestamp to every mempool entry. Run a scheduled cleanup thread every 10 minutes using `ScheduledExecutorService`. It iterates the mempool and removes any transaction older than your TTL (start with 24 hours). Also in `mineBlock()`, when a transaction fails state validation, call `mempool.remove(txId)` on it instead of throwing — skip it, log it, and continue with the next transaction. The block gets mined without the zombie transaction and the zombie is cleaned up.

Issue 19 — Self-resolving disputes

In `InvoiceRecord`, add a `resolverBusinessId` field. In the `RESOLVE_DISPUTE` case in `StateMachine`, extract the sender's `businessId` from the transaction and compare it against both `r.issuerBusinessId` and `r.counterpartyBusinessId`. If it matches either, throw `StateTransitionException("dispute parties cannot self-resolve")`. That's it — four lines of code that close the entire attack.

Issue 20 — Fixed difficulty

~~Add a difficulty adjustment check in `Node.mineBlock()`. Every 2,016 blocks (Bitcoin's adjustment interval, roughly two weeks at 10-minute blocks), compare actual elapsed time to target elapsed time. If blocks came in twice as fast as intended, double the difficulty. If twice as slow, halve it. Cap the adjustment at 4x up or down per period to prevent wild swings. Store the adjustment target block time as a node configuration parameter.~~

Issue 21 — Merkle CVE-64

In `Merkle.java`, change the odd-list handling. Instead of duplicating the last element, append `HashUtil.sha256("")` — a hash of an empty string — as the padding node. Now `[A,B,C,D,E]` and `[A,B,C,D,E,E]` produce different roots because the padding is a fixed known value, not a copy of E.

Storage Problems

Issue 22 — Height cast to int

Replace every `(int)height` cast with `Math.toIntExact(height)`. This throws a clear `ArithmeticException` with a meaningful message if the value ever overflows, instead of silently wrapping to a garbage negative number. Long term, redesign the offset list to use a `LongToLongHashMap` from Eclipse Collections or a memory-mapped file index to support chains of any size.

Issue 23 & 24 — Charset bugs on write and read

These two are fixed together. On the write side, replace `new FileWriter(logFile.toFile(), true)` with `new OutputStreamWriter(new FileOutputStream(logFile.toFile(), true), StandardCharsets.UTF_8)`. On the read side, replace `raf.readLine()` with: read a line as raw bytes using `raf.read(buffer)` until you hit a newline byte, then decode with `new String(buffer, StandardCharsets.UTF_8)`. Both sides now explicitly use UTF-8 regardless of what OS or locale the server is running on.

Issue 25 — No fsync

After the write and flush in `BlockStore.append()`, add one line: `FileDescriptor.sync()` called on the underlying file descriptor, or switch to `FileChannel` and call `channel.force(true)`. This tells the OS to flush its write buffer to physical disk before your method returns. Only after that sync completes do you update the offset index. Now a power failure between the write and the sync loses nothing — the partial write is detected on restart and the index rebuilds correctly from the last complete line.

Python Analytics Problems

Issue 26 — Double-counted disputes

Replace the two-condition dispute count with a single check on a field that's set once and never changes. Add a boolean `ever_disputed` column to your analytics — True if the invoice ever entered DISPUTED state, regardless of what status it is now. Count that instead of current status. One line change in the aggregation function.

Issue 27 — Dead concentration risk code

Implement it properly or delete it. If implementing: for each business, calculate the percentage of total invoice volume going to their single largest trading partner. Add it to the risk score with a weight (something like $10 \times \text{concentration_ratio}$ where ratio is 0 to 1). If deleting: remove the `counter_col` line and add a comment in the risk report output noting that concentration risk is a planned v2 metric. Don't leave dead code that implies a calculation is happening when it isn't.

Issue 28 — Fragile numpy truthiness

Find every `if paid` and `if count` style guard in the analytics script. Replace every one with `if paid > 0` and `if count > 0`. This is explicit, type-safe, and works correctly regardless of what numeric type pandas returns. Ten-minute fix across the whole file.

Issue 29 — Unweighted risk score

Change every rate calculation from count-based to amount-based. Instead of `late_count / total_count`, calculate `late_amount_cents / total_amount_cents`. Instead of `paid_count / total_count`, use `paid_amount_cents / total_amount_cents`. Add `amount_cents` to your aggregation groupby. The risk score now reflects financial exposure, not just transaction frequency. A business that paid one \$5M invoice late is correctly scored as riskier than one that paid fifty \$10 invoices late.

Issue 30 — daysToPay from wrong timestamp

In the analytics script, change the `daysToPay` calculation from `paidAt - issuedAt` to `paidAt - acceptedAt`. Make sure `acceptedAt` is captured in your `InvoiceRecord` when the `ACCEPT_INVOICE` state transition fires in `StateMachine` — add `r.acceptedAt = tx.timestampEpochSec` in that case. The clock now starts when the buyer acknowledges the obligation, not when the seller created the invoice. Every risk score in the system immediately becomes more accurate.

Priority order to fix these

If you're doing this yourself, tackle them in this sequence:

First fix issues **7 and 8** — TLS and authentication. Every other fix is pointless if anyone on the network can steal keys or spam the node.

Then fix **1, 2, 3** — the critical bugs that cause data loss and downtime.

Then **14, 15, 18** — the concurrency and mempool issues that cause mining failures and memory leaks under real load.

Then **6, 10, 19** — the replay and dispute security holes.

Then **16** — fork handling, which is the largest single piece of work.

Then **20, 17, 21** — difficulty adjustment, empty block prevention, Merkle fix.

Then the storage fixes **22, 23, 24, 25** — these can be done in one session since they're all in `BlockStore.java`.

Then the analytics fixes **26, 27, 28, 29, 30** — one afternoon in Python.

Then the remaining cleanup **4, 5, 11, 12, 13** — these are lower risk but should be done before you onboard any real clients.

New issues PoA introduces that PoW didn't have

Validator key compromise is catastrophic. In PoW, if someone gets your hardware they can mine blocks but the rest of the network can out-hash them. In PoA, if someone steals a validator's private key they can sign any block they want and the network accepts it as legitimate. You now have a small set of extremely high-value keys that need serious operational security — hardware security modules (HSMs), cold storage, strict access controls. This is the single biggest new risk PoA introduces.

Validator collusion is the new 51% attack. In PoW, you need 51% of hashrate to attack the chain. In PoA, if you have 3 validators and 2 collude, they control the chain. Your validator set size and selection process is now your primary security parameter. For a pilot with trusted parties this is fine. For a public network it needs careful governance.

You need a validator onboarding and offboarding process. What happens when you want to add a new validator (a new trusted anchor buyer, a new regional partner)? What happens when you need to remove one (a validator goes offline, a business leaves the network, a key is compromised)? This needs a defined governance process — either a smart contract vote, a multisig admin key, or a manual config update with a coordinated restart. None of this exists in your current code.

Liveness depends on validators being online. In PoW, anyone can mine at any time. In PoA with round-robin, if the validator whose turn it is goes offline, block production stalls until their slot passes and the next validator takes over. You need a timeout mechanism — if the expected validator doesn't produce a block within N seconds, the next validator in the list takes over for that slot.

The `POST /mine` endpoint needs authentication more urgently than before. In PoW, an unauthorized caller triggering `/mine` just wastes CPU. In PoA, an unauthorized caller triggering block production with a stolen validator key produces legitimately valid blocks the whole network accepts. The authentication fix from issue 8 is now critical, not just important.

Bottlenecks:

Biggest bottleneck — everything goes through one Java process

Your entire system is one JVM. The REST API, the P2P network, the state machine, the block production, and the disk writes all share the same process, the same heap, and the same thread pool. There is no horizontal scaling possible right now. When that process is under load, everything degrades together. One expensive operation — say, a large block with 500 transactions being validated — slows down every concurrent API call from every user simultaneously.

Throughput bottleneck — sequential block production

In PoA with round-robin, only one validator produces a block at any given slot. That validator builds the block, signs it, writes it to disk, then broadcasts it. The next block cannot start until peers receive and accept the previous one. This means your maximum transaction throughput is:

$\text{max TPS} = \text{maxTxPerBlock} / \text{targetBlockTimeSeconds}$

If you allow 100 transactions per block and produce a block every 5 seconds, your ceiling is 20 transactions per second. At \$50M monthly volume with an average invoice of \$25,000, you're processing roughly 2,000 invoices per month — less than 1 per minute. That's fine today. But each invoice generates multiple transactions (ISSUE, ACCEPT, CONFIRM, PAY = 4 transactions minimum per invoice). So 2,000 invoices = 8,000+ transactions per month, still well within a 20 TPS ceiling. The bottleneck won't hurt you at pilot scale but you need to know it exists.

Disk I/O bottleneck — append-only log with no buffering

Every transaction that gets mined triggers a synchronous write to `blocks.log`. With the `fsync` fix from issue 25, that write now also waits for the OS to confirm the bytes hit physical storage before continuing. On a standard SSD, one `fsync` takes 1-10 milliseconds. That's fine for low volume. But if you're doing 20 TPS and each block requires one `fsync`, and block production is every 5 seconds, you're fine. The problem comes if you ever increase block frequency to sub-second intervals — then `fsync` latency becomes your hard ceiling on throughput.

The deeper problem is that the entire node state is rebuilt from this log on every restart. Right now replaying 10,000 blocks takes seconds. At 1,000,000 blocks it takes minutes. At 10,000,000 blocks it takes hours. You have no checkpoint mechanism — no way to snapshot state at block N and start replay from there instead of from genesis.

Memory bottleneck — the mempool and state live entirely in RAM

Your `State` object (all invoice records) and your `Mempool` (all pending transactions) live in a `ConcurrentHashMap` in the JVM heap. There is no eviction from `State` — every invoice ever created stays in memory forever. At pilot scale with 200 SMEs this is negligible. But if `SMEChain` scales to 10,000 businesses with 100 invoices each, that's 1,000,000 `InvoiceRecord` objects in memory permanently. Java object overhead means this could consume several gigabytes of heap, triggering frequent GC pauses that freeze the entire node.

Network bottleneck — broadcast is $O(n)$ TCP connections per message

Every time your node broadcasts a block or transaction, it opens a new TCP connection to every peer, sends one message, and closes the connection. With 3 peers this is fine. With 20 peers every broadcast opens 20 TCP connections simultaneously. With 100 peers it opens 100. TCP connection setup has overhead — handshake, kernel resources, port allocation. Under high transaction volume with many peers, the broadcast loop becomes a bottleneck and starts failing silently (errors are swallowed) because peer connections time out faster than they can be opened.

Validator bottleneck — single point of liveness per slot

In your round-robin PoA, if the validator whose turn it is goes offline or is slow, block production stalls for that entire slot duration. If your slot is 5 seconds and the validator is offline, every transaction submitted during that 5 seconds sits in the mempool waiting. If the validator is offline for 10 minutes, nothing moves for 10 minutes. There is no fallback in the current code — no timeout after which the next validator takes over automatically.

State machine bottleneck — full replay on every block

When `tryAddBlock()` is called, the state machine re-applies every transaction in the block against a copy of the current state. For small blocks this is instant. But `StateMachine.apply()` is called once per transaction, and each call does map lookups, type switches, and object mutations. If you increase block size to 10,000 transactions to improve throughput, validating one block could take hundreds of milliseconds — during which the write lock is held and the node is frozen.

REST API bottleneck — single-threaded request handling

Java's built-in `HttpServer` (which you're using) uses a single executor to handle requests. Under concurrent load from multiple SMEs submitting invoices simultaneously, requests queue up behind each other. The first request to hit a slow operation (like `GET /export/snapshot` which serializes the entire chain state to JSON) blocks every other request until it completes.

The practical priority

At your current target of 200 SMEs and \$50M monthly volume, none of these bottlenecks will actually hurt you. The math works out comfortably within your current architecture's limits.

The ones to solve before you scale beyond that are, in order:

First, add a **checkpoint/snapshot mechanism** to `BlockStore` so you don't replay from genesis on every restart — this is the one that will bite you earliest as the chain grows.

Second, replace Java's built-in `HttpServer` with **Jetty or Undertow** — a proper async HTTP server that handles concurrent requests without queuing them behind each other.

Third, add a **validator timeout and failover** mechanism so one offline validator doesn't stall the whole chain.

Fourth, move the **State object to a proper database** (Postgres or RocksDB) rather than keeping it all in the JVM heap — this is what unblocks you from the memory ceiling.

Everything else is fine to leave until you're past \$50M volume and actually feel the pressure.